

Android横竖屏切换全解析及第三方适配方案

在Android应用开发中，横竖屏切换是常见的交互需求，尤其在视频播放、阅读、游戏等场景中至关重要。然而，屏幕方向切换会触发Activity重建、布局重绘等一系列操作，若处理不当易导致数据丢失、UI错乱等问题。本文将从核心原理出发，详解原生配置方法、生命周期影响及解决方案，并对比主流第三方适配库的优势，为开发者提供完整的技术指南。

一、横竖屏切换核心原理

Android系统中，屏幕方向属于“配置变更”（Configuration Changes）的一种。当用户旋转屏幕时，系统会检测到屏幕尺寸、宽高比等配置信息变化，默认行为是销毁当前Activity并重新创建实例，以此加载与新方向匹配的资源（如布局、图片）。这一机制的核心目的是保证应用在不同方向下的显示适配，但也给开发者带来了状态保存与恢复的挑战。

需要注意的是，Android 3.2（API 13）后引入了“最小宽度”（smallestWidth）等配置，屏幕旋转引发的配置变更维度更丰富，开发者需结合系统版本进行适配。

二、原生横竖屏配置方式（Java/XML）

原生开发中，横竖屏切换的控制主要通过清单文件（AndroidManifest.xml）配置和Java代码动态设置实现，两种方式可根据需求组合使用。

1. XML静态配置（基础方式）

在AndroidManifest.xml的Activity节点中，通过 `android:screenOrientation` 属性指定屏幕方向，常用取值如下：

- `portrait`：固定竖屏，禁止切换
- `landscape`：固定横屏（默认横向，通常为左横屏）
- `reverseLandscape`：反向横屏（右横屏）
- `sensor`：跟随传感器，允许自动切换（默认行为）
- `user`：跟随用户设置的方向

示例：固定Activity为竖屏并禁止切换

Code block

```
1 <activity
2     android:name=".MainActivity"
3     android:screenOrientation="portrait">
4     <intent-filter>
```

```
5         <action android:name="android.intent.action.MAIN" />
6         <category android:name="android.intent.category.LAUNCHER" />
7     </intent-filter>
8 </activity>
```

同时，若需在不同方向加载不同布局，可在res目录下创建 `layout-land`（横屏）和 `layout-port`（竖屏）文件夹，系统会自动根据屏幕方向匹配对应的布局文件（如activity_main.xml）。

2. Java动态控制（灵活适配）

通过代码动态设置屏幕方向，可实现根据业务场景切换方向（如视频播放时自动横屏，退出播放后恢复竖屏），核心API为 `setRequestedOrientation()`。

示例：按钮控制横竖屏切换

Code block

```
1 public class MainActivity extends AppCompatActivity {
2     private Button btnSwitch;
3     private boolean isPortrait = true;
4
5     @Override
6     protected void onCreate(Bundle savedInstanceState) {
7         super.onCreate(savedInstanceState);
8         setContentView(R.layout.activity_main);
9
10        btnSwitch = findViewById(R.id.btn_switch);
11        btnSwitch.setOnClickListener(v -> {
12            if (isPortrait) {
13                // 切换为横屏
14
15                setRequestedOrientation(ActivityInfo.SCREEN_ORIENTATION_LANDSCAPE);
16            } else {
17                // 切换为竖屏
18
19                setRequestedOrientation(ActivityInfo.SCREEN_ORIENTATION_PORTRAIT);
20            }
21            isPortrait = !isPortrait;
22        });
23    }
24 }
```

动态设置的优先级高于XML配置，适合需要临时变更方向的场景。此外，可通过 `getRequestedOrientation()` 获取当前屏幕方向。

三、横竖屏切换对生命周期的影响及解决方案

如前文所述，默认情况下屏幕旋转会销毁并重建Activity，生命周期回调顺序为：`onPause()` → `onStop()` → `onDestroy()` → `onCreate()` → `onStart()` → `onResume()`。这会导致内存中临时数据（如输入内容、列表滚动位置）丢失，需通过以下两种核心方案解决。

1. 状态保存与恢复（原生方案）

利用Activity的 `onSaveInstanceState()` 和 `onRestoreInstanceState()` 方法保存与恢复数据，前者在Activity销毁前调用，后者在重建后调用。

Code block

```
1  @Override
2  protected void onSaveInstanceState(Bundle outState) {
3      super.onSaveInstanceState(outState);
4      // 保存临时数据（如输入框内容）
5      EditText etInput = findViewById(R.id.et_input);
6      outState.putString("input_content", etInput.getText().toString());
7      // 保存列表滚动位置
8      ListView lv = findViewById(R.id.lv_content);
9      outState.putInt("list_position", lv.getFirstVisiblePosition());
10 }
11
12 @Override
13 protected void onRestoreInstanceState(Bundle savedInstanceState) {
14     super.onRestoreInstanceState(savedInstanceState);
15     // 恢复数据
16     String inputContent = savedInstanceState.getString("input_content");
17     EditText etInput = findViewById(R.id.et_input);
18     etInput.setText(inputContent);
19
20     int listPosition = savedInstanceState.getInt("list_position");
21     ListView lv = findViewById(R.id.lv_content);
22     lv.setSelection(listPosition);
23 }
```

需注意，Bundle仅能存储可序列化数据（如基本类型、String、Parcelable对象），复杂数据建议通过ViewModel或单例模式管理。

2. 禁止Activity重建（高效方案）

在XML配置中通过 `android:configChanges` 属性声明需要自行处理的配置变更，系统将不再销毁Activity，而是调用 `onConfigurationChanged()` 方法通知开发者处理变更。

示例：声明处理屏幕方向和屏幕尺寸变更

Code block

```
1 <activity
2     android:name=".MainActivity"
3     android:configChanges="orientation|screenSize|smallestScreenSize">
4 </activity>
```

对应Java代码中处理变更逻辑：

Code block

```
1 @Override
2 public void onConfigurationChanged(Configuration newConfig) {
3     super.onConfigurationChanged(newConfig);
4     // 判断新方向
5     if (newConfig.orientation == Configuration.ORIENTATION_LANDSCAPE) {
6         Toast.makeText(this, "切换为横屏", Toast.LENGTH_SHORT).show();
7         // 手动更新布局或数据
8         updateLayoutForLandscape();
9     } else if (newConfig.orientation == Configuration.ORIENTATION_PORTRAIT) {
10        Toast.makeText(this, "切换为竖屏", Toast.LENGTH_SHORT).show();
11        updateLayoutForPortrait();
12    }
13 }
```

该方案避免了Activity重建，效率更高，但需手动处理布局调整、资源加载等逻辑，适合复杂UI场景。

四、第三方适配库及选型建议

原生方案在复杂应用中存在代码冗余、状态管理繁琐等问题，第三方适配库通过封装通用逻辑，简化横竖屏切换的开发流程。以下为两款主流库的对比与使用指南。

1. AutoSize（今日头条适配方案）

AutoSize是今日头条团队开源的屏幕适配库，核心优势在于“零侵入式”适配横竖屏切换，无需修改布局文件，自动适配不同方向的屏幕尺寸。

使用步骤：

Code block

```
1 // 引入依赖
2 implementation 'me.jessyan:autosize:1.2.1'
```

在Application中初始化（可选，默认已适配）：

Code block

```
1 public class MyApp extends Application {
2     @Override
3     public void onCreate() {
4         super.onCreate();
5         AutoSize.initCompatMultiProcess(this);
6     }
7 }
```

AutoSize支持在XML中通过注解指定适配规则，横竖屏切换时自动重新计算控件尺寸，避免UI变形。适合对布局精度要求高的应用（如电商、新闻）。

2. ImmersiveMode（沉浸式+横竖屏适配）

ImmersiveMode专注于沉浸式体验与横竖屏切换的结合，解决了切换时状态栏、导航栏的适配问题，常见于视频、游戏类应用。

核心功能：

- 横竖屏切换时自动保持沉浸式状态
- 提供方向切换监听回调，简化业务逻辑
- 兼容Android 4.4+所有版本

Code block

```
1 // 初始化沉浸式与横竖屏监听
2 ImmersiveMode.with(this)
3     .enableOrientationSwitch()
4     .setOrientationChangeListener(orientation -> {
5         if (orientation == ImmersiveMode.LANDSCAPE) {
6             // 横屏逻辑（如隐藏标题栏）
7             getSupportActionBar().hide();
8         } else {
9             getSupportActionBar().show();
10        }
11    })
12    .init();
```

3. 选型对比与建议

库名称	核心优势	适用场景	缺点
AutoSize	布局适配精准，零侵入	电商、新闻、阅读类应用	仅专注尺寸适配，

ImmersiveMode	沉浸式+方向切换联动	视频、游戏、全屏应用	功能单一，需配合数据
原生方案	无依赖，兼容性强	简单场景或轻量级应用	复杂场景代码冗余

五、总结

横竖屏切换适配的核心是平衡“系统配置变更”与“应用状态稳定性”。原生开发中，静态配置适合固定方向需求，动态设置适合灵活场景，而 `configChanges` 属性可避免Activity重建；第三方库中，AutoSize专注布局适配，ImmersiveMode擅长沉浸式联动，开发者需根据业务场景选型。

实际开发中，建议结合ViewModel（存储数据）+ AutoSize（布局适配）的组合方案，既保证状态稳定，又简化适配流程，为用户提供流畅的横竖屏交互体验。

（注：文档部分内容可能由 AI 生成）